Logging

Objectives

- Explain why a program logs instead of printing with System.out
- Choose a log level for an event, from debug to error
- Log an exception with its stack trace and without sensitive data

Teaching

This code uses an SLF4J logger, the logging API that Javalin projects use. The logger is set to show messages at level INFO and above. Predict before any explanation: which lines appear?

```
log.debug("cart has {} items", 3);
log.info("order {} placed", "A-1001");
log.warn("payment service slow, retry {} of {}", 1, 3);
```

```
private static final Logger log = LoggerFactory.getLogger(Checkout.class);
log.info("order {} placed", orderId);   // {} is replaced by orderId
```

SLF4J's levels, from most to least detailed:

- TRACE and DEBUG: details that help a developer follow what the code did.
- INFO: normal milestones, such as a server starting or an order being placed.
- WARN: something unexpected that the program handled, such as a retry.
- ERROR: an operation failed.

With a threshold of INFO, INFO, WARN and ERROR are written; DEBUG and TRACE are dropped.

To log a failure with its stack trace, pass the exception itself as the last argument. SLF4J then prints the stack trace after the message. Passed anywhere else, or as `e.getMessage()`, it is only text.

```
log.error("could not save invoice {}", invoiceId, e);
```

Logs are copied to files and monitoring tools and read by many people. OWASP lists data to keep out of them, including passwords, session ids, access tokens and payment card data. Identify what failed by a

safe id, such as an order number.

`System.out.println` writes every line the same way, every time, to one place. A logger separates what the code reports from how it is handled: the code says the level and the message, and configuration decides the threshold, the format, such as time and class name, and the destination, such as the console or a file. Changing any of those needs no code change.

Model

The model: every log message carries a level, and configuration sets a threshold. Messages at or above the threshold are written; the rest are dropped without changing the code.

Recap

- A logger attaches a level to each message; configuration sets the threshold, format and destination without code changes.
- Levels from most to least detailed: TRACE, DEBUG, INFO, WARN, ERROR.
- Pass the exception as the last argument to log its stack trace.
- Keep passwords, tokens, session ids and payment card data out of logs; identify things by safe ids.

Next module: HTTP and Javalin

Cheat sheet

Where to act: The event shown after Event:, above the four level buttons.

Next step: Ask who needs the message and when: only a developer tracing code, anyone watching normal operation, someone who should know a problem was handled, or someone who must act because something failed.

Where to act: The last line in the editor, the `System.out.println` call.

Next step: The payment failed and the order was not placed. Pick the matching level method on `log`, and decide which of the two variables in the comments is safe to show.

Another hint Another hint, 2 of 2 for Activity 3

Where to act: Inside the parentheses of your `log` call.

Next step: Write a message with one `{}` for the safe id, then list the arguments in order: the id for the `{}`, and after it the caught exception variable itself, not its message.

A ship's crew writes every event in a logbook and marks each entry as routine, notable or urgent. At sea the captain reads only the urgent entries each hour; after a storm the engineers read every routine entry too. Nobody rewrites the logbook for each reader; each reader just chooses which marks to read.

Levels are the marks, and configuration is each reader choosing which marks to see.

Create	`private static final Logger log = LoggerFactory.getLogger(Checkout.class);`
Placeholder	`log.info("order {} placed", orderId);`
With stack trace	`log.error("could not save invoice {}", invoiceId, e);`
Keep out	passwords, tokens, session ids, card numbers

Sources

Claims on this page were checked against the live pages below on 6 November 2026 (UTC). Logger output was confirmed with SLF4J 2.0.17 and its simple backend on OpenJDK 25.0.4.1 (Temurin). Links open outside this lesson.

- Javalin documentation, Logging: Javalin uses SLF4J and requires an SLF4J implementation.
- SLF4J user manual: Logger, LoggerFactory and `{}` placeholders.
- SLF4J FAQ: an exception passed as the last argument is logged with its stack trace.
- OWASP Logging Cheat Sheet: data to exclude from logs.

Static interaction alternatives

predict

Prediction (practice, not scored): a logger set to INFO receives a debug, an info and a warn call. Options: all three lines (wrong: DEBUG is below the INFO threshold); only the INFO and WARN lines, with {} replaced by the arguments (correct); only the WARN line (wrong: INFO is at the threshold, so it is shown); the INFO and WARN lines with the {} left as typed (wrong: each {} is replaced by the next argument).

drag

Sorting activity: server started and listening on port 7070 is INFO. Computed discount 0.10 for cart 98 from 3 rules is DEBUG. Payment service timed out, retrying, attempt 2 of 3 is WARN. Could not save order A-1001, so the customer saw an error page, is ERROR. User 1042 signed in is INFO. Configured theme value is invalid, so the service uses the default, is WARN. The values compared on each pass of a matching loop are DEBUG.

explore

Java editor on one statement inside catch (PaymentException e), where orderId is A-1001 and cardNumber is 1234-5678-9012-3456 and the logger threshold is INFO. Run shows what the line writes and checks four things: the level is ERROR; A-1001 appears; the card number does not; and the stack trace follows the message, starting with PaymentException: card declined. The starter println has no level, prints the card number and prints only the message text. A debug call writes nothing at an INFO threshold. Passing e.getMessage() logs card declined as text but no stack trace. A {} with no argument left for it is printed as {}.

mcq

Apply question: switching from println to a logger gives each message a level, while configuration chooses the threshold, format and destination without code changes (correct). Wrong: faster code because a logger never builds strings (formatting still happens for written messages); nothing, because println output can be filtered the same way (println lines have no level to filter on); automatic removal of secrets (a logger writes whatever it is given). Retry form: keep the debug calls in the code, run production at INFO and switch to DEBUG when investigating.